Fitness Tracker

Silver Gjeka

University: University of Verona (UNIVR)
Subject: Machine Learning
Degree: Master's Degree in Artificial Intelligence
Matriculated: VR527645

Introduction

Fitness trackers are now on many people's wrists, helping them count steps, check heart beats, and track sleep. They've become needed tools in daily life because they help us understand our health better. With a quick look at your wrist, you can see if you have moved enough today or if your heart is working too hard. Trackers remind busy people to stand up and move when they've sat too long. They also help to show patterns over time, such as if you sleep better on days when you exercise. For people who are trying to be healthier, these devices provide the numbers and facts that keep them going. In this project, we will see how to track various gym exercises like bench press, squats, and more.

Machine Learning Role

Machine learning helps users automatically track their gym workouts by recognizing different exercises through motion data from sensors such as accelerometers and gyroscopes. Instead of manually logging exercises, users get real-time exercise detection. This makes it easier to stay consistent, monitor progress, and stay motivated throughout your fitness journey.

Dataset Description

The dataset consists of motion sensor data from 5 participants performing a variety of fitness exercises. The data includes accelerometer and gyroscope readings, with each set labeled according to the specific exercise. The data set covers six different exercises, including bench press, overhead press, squat, deadlift, row, and rest, and participants perform varying numbers of sets for each exercise.

The dataset includes the following variables:

- **Epoch (ms):** Millisecond timestamp of each sensor reading.
- **Accelerometer_x/y/z:** Measures body acceleration in three directions.
- **Gyroscope_x/y/z:** Measures rotational velocity in three directions.
- **Participant:** The person performing the exercise.
- **Label:** The specific exercise performed (e.g., squat).
- **Category:** Intensity level (e.g., light, medium, heavy).
- **Set:** Identifier for each repetition set.

Objective

The objective of this project is to develop a machine learning model capable of recognizing and classifying gym exercises in real time using sensor data from accelerometers and gyroscopes. To achieve this, we will experiment with various models like Support Vector Machines (SVM), Random Forest and others, to identify which approach offers the best performance in predicting exercises. One challenge is making sure the model works well for different people, exercise types, and movement styles, so it can be used by anyone.

Studying the Dataset

In this project, we work with real-time movement data collected every 200 milliseconds (epoch) using wearable devices. These devices include two main sensors: an accelerometer and a gyroscope. The dataset includes data from multiple participants, each performing a variety of exercises from different categories. This variety helps train machine learning models to recognize different movement patterns across different people.

Accelerometers: measure how fast something is speeding up or slowing down in three directions: up/down,left/right, and forward/backward (X, Y, Z axes). This helps capture simple body movements like walking, jumping, or lifting weights. When you move your arm or leg, the accelerometer picks up that motion as changes in speed.

Gyroscopes: measure how something is rotating or turning around the same three directions. They track rotation and balance, such as when you twist your body, bend forward, or turn to the side. This helps detect more complex movements that involve changes in body position.

Together, these sensors give us a full picture of what the body is doing both in terms of straight movement and rotation. By using the signals they produce, we can train a machine learning model to recognize which exercise a person is performing at any moment.

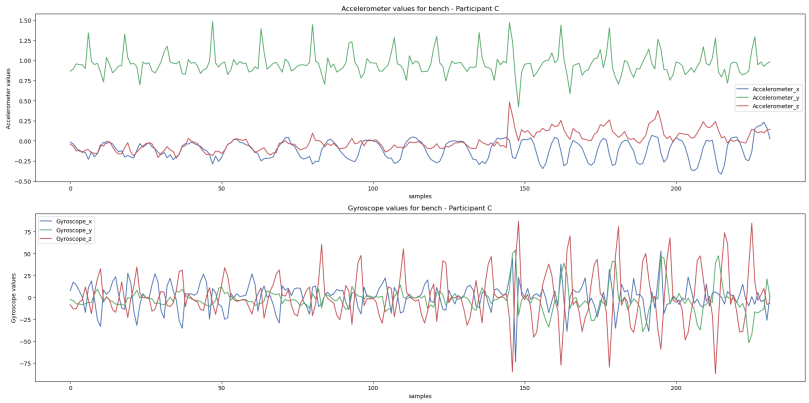

Figure 1: Accelerometer and Gyroscope Data from Participant.

Why This Dataset?

This dataset is a time series, it shows how movements change over time. This is great for tracking exercises. It has detailed data from accelerometers and gyroscopes, so we can see both straight and turning movements. The data is recorded very often, which makes it good for real-time predictions. It also helps learning how to work with signals. For example, trying things like smoothing the data or using sliding windows to get it ready for machine learning.

Data Preprocessing

Before using the sensor data, we need to clean it to make sure it's accurate and useful. One common problem is outliers unusual values caused by sensor errors, random noise, or sudden unexpected movements. By correcting these outliers, we avoid confusing the machine learning models and improve the accuracy.

Outliers

We analyze accelerometer and gyroscope data during exercises. Figures 2 show the boxplots.

Gyroscope: For example in a squat, the body moves mostly up and down, not much turning. So the gyroscope values stay close to zero. Even a small twist shows up as an outlier because it stands out from the rest of the data. This is why we see many outliers the normal values are tightly grouped, so small changes look big.

Accelerometer: Squats involve big up-and-down movements, which the accelerometer captures. The median line may not be in the center, showing that the motion isn't perfectly balanced. But since large changes are expected, we see fewer outliers.

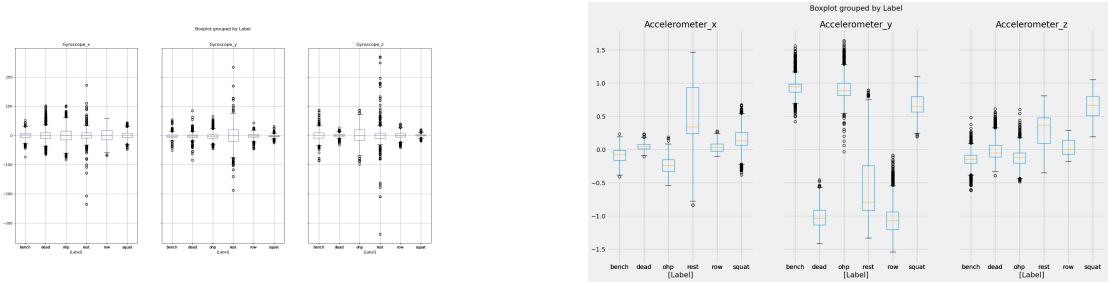

Figure 2: Outliers detected Gyroscope, Accelerometer.

Detecting Outliers with DBSCAN

To detect outliers, using **DBSCAN** algorithm, that is a clustering method that finds groups of similar data points based on how close they are to each other. It works well for accelerometer and gyroscope data because it doesn't expect the data to follow a normal shape. Instead, it looks for dense areas in the data and marks anything far from these areas as an outlier. This makes it great for detecting outliers in sensor data, which often varies a lot.

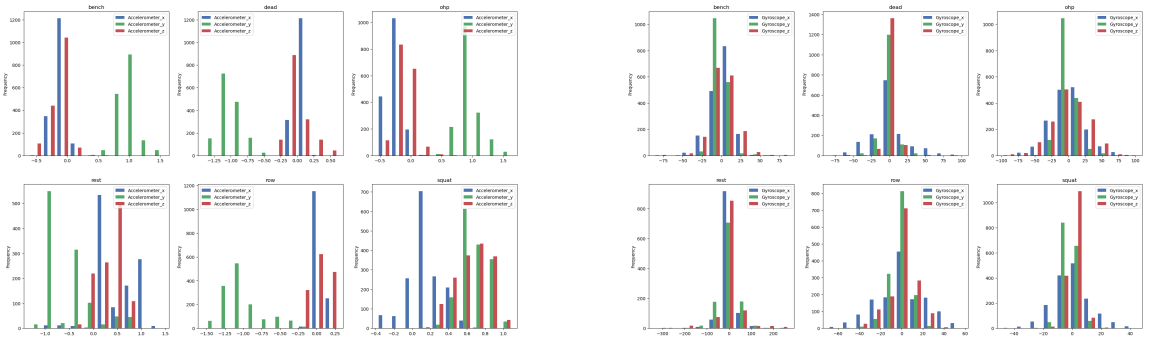

Figure 3: Normal Distribution in Gyroscope, Accelerometer.

As shown in the figure, our sensor data doesn't follow a normal (bell-shaped) distribution.

Dealing with Outliers

After detecting outliers using **DBSCAN**, we do not want to remove them to avoid introducing gaps in the time-series data. Instead, we used **linear interpolation** to estimate and replace the outlier values.

The process worked as follows:

- DBSCAN identified outliers and labeled them with `-1`.
- For each selected feature (e.g., accelerometer and gyroscope data), we retained the original values and applied interpolation only where outliers were detected.
- **Linear interpolation** estimates the missing or noisy values by drawing a straight line between the values immediately before and after the outlier, filling in the gap with a value along that line.

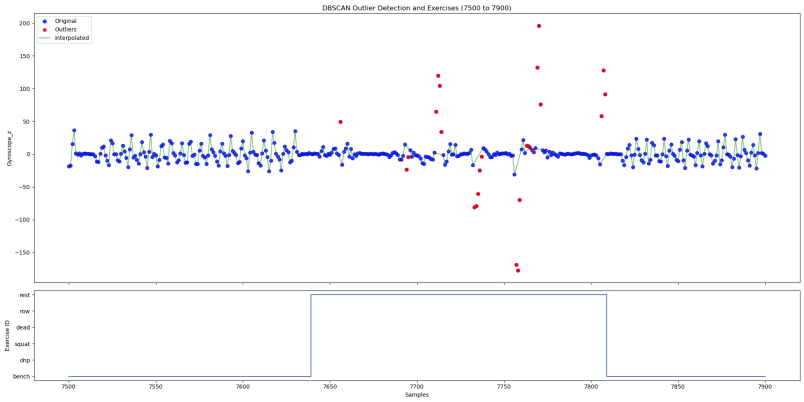

Figure 4: Corrected Signal Using Interpolation After Outlier Removal.

Feature Engineering

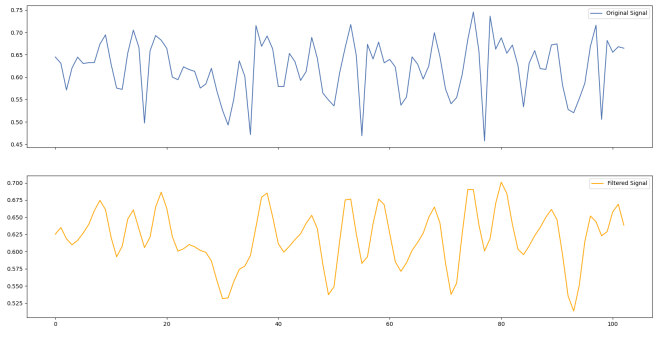

Figure 5: Transformation before and after the filter.

In the feature engineering step, is applied a low-pass filter to smooth the sensor data. The low-pass filter removes unwanted high-frequency noise, like sensor errors and vibrations, while keeping the important slower signals that show real body movement.

The cutoff frequency to 1.0 Hz to keep the slower movements from exercises, like squats, and remove faster noise. By removing the noise, the data becomes easier to analyze, leading to more accurate results.

Feature Extraction

1. Principal Component Analysis (PCA)

Before applying PCA, the data was normalized to ensure a fair comparison between larger and smaller values.

PCA was then applied to reduce the number of features for the gyroscope and accelerometer data. Using the elbow technique, the original six columns were reduced to three principal components: `pca_1`, `pca_2`, and `pca_3`.

These three PCA components were added to the dataset to see how well this technique perform in this case.

2. Magnitude Calculation

To measure the overall strength of motion, we compute the magnitude of the accelerometer and gyroscope signals using the Euclidean norm. This combines the x, y, and z components into a single value:

$$\text{Magnitude} = \sqrt{x^2 + y^2 + z^2}$$

This helps summarize 3D motion into one value for easier analysis, such as classifying different exercises.

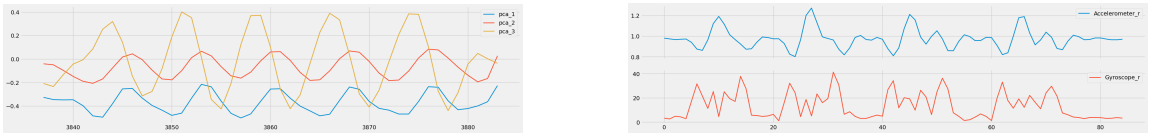

Figure 6: Extracted PCA components and squared magnitudes.

3. Abstract Mean

To make the sensor data smoother and easier to understand, a **moving average** is used. It takes a few values next to each other and finds their average, making the data smoother and less jumpy.

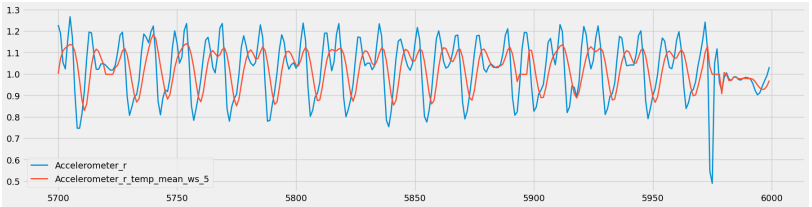

Figure 7: Extracting the abstract mean.

A **window size of 5** is used, meaning every 5 values are averaged together. This reduces noise and shows the overall trend in movement.

The moving average is applied to all sensor values, including the **accelerometer** and **gyroscope** magnitudes, for every data set.

NaN (missing) values can appear at the start, end, or even within the signal. At the start and end, there may be too few values to fill the window. Within the signal, NaNs occur if the original data contains missing values that affect the average calculation.

3.1 Handling NaN Values

To fix this, NaNs are replaced by the mean of each column, calculated while ignoring NaNs. This keeps the data smooth and complete without gaps.

Clustering

k-Means clustering is applied to both accelerometer and gyroscope data to identify patterns in motion signals. The number of clusters is chosen to match the number of exercise types, aiming to group similar movements together.

In Figure 8, each color represents a different cluster, showing how motion data points are separated based on their characteristics. These clusters are then used as labels for training the model to recognize specific types of exercises.

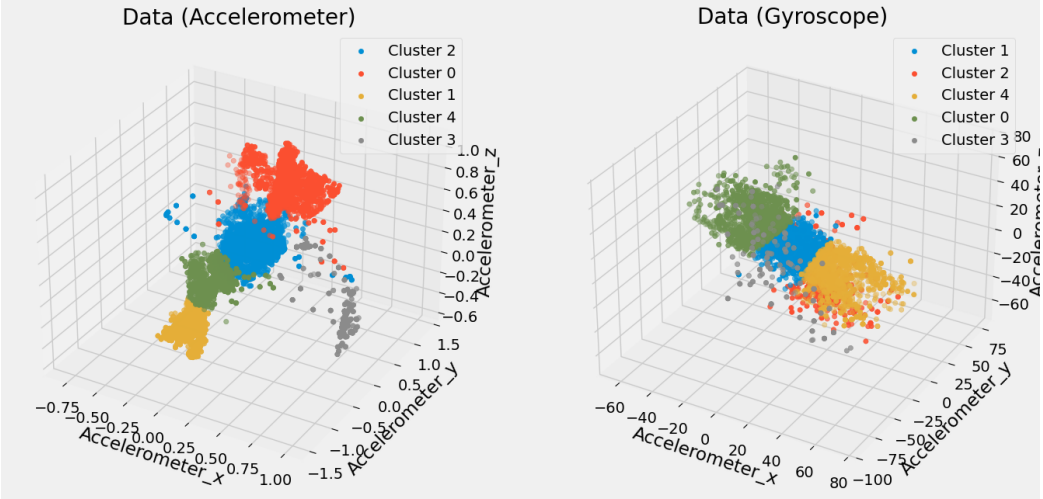

Figure 8: Clustering of accelerometer and gyroscope data.

Feature Correlation

Feature correlation was analyzed to identify variables that are strongly related to each other and to the target behavior.

The heatmap in Figure 9 shows the Pearson correlation coefficients between all extracted features. High correlation values (close to 1 or -1) indicate a strong linear relationship between two features, while values near 0 suggest weak or no correlation.

Two clear examples can be observed:

- `Accelerometer_x` and `Accelerometer_x_temp_mean_ws_5` have a very strong positive correlation ($r = 0.95$), since the temporal mean is directly computed from the original signal.
- `pca_1` shows high correlation with several original sensor features, indicating that the first principal component effectively captures a large portion of the data's variance.

Additionally, the clustering labels (`cluster_acc`, `cluster_gyro`) show moderate correlations with several features, confirming that the clustering process captures meaningful patterns in the data.

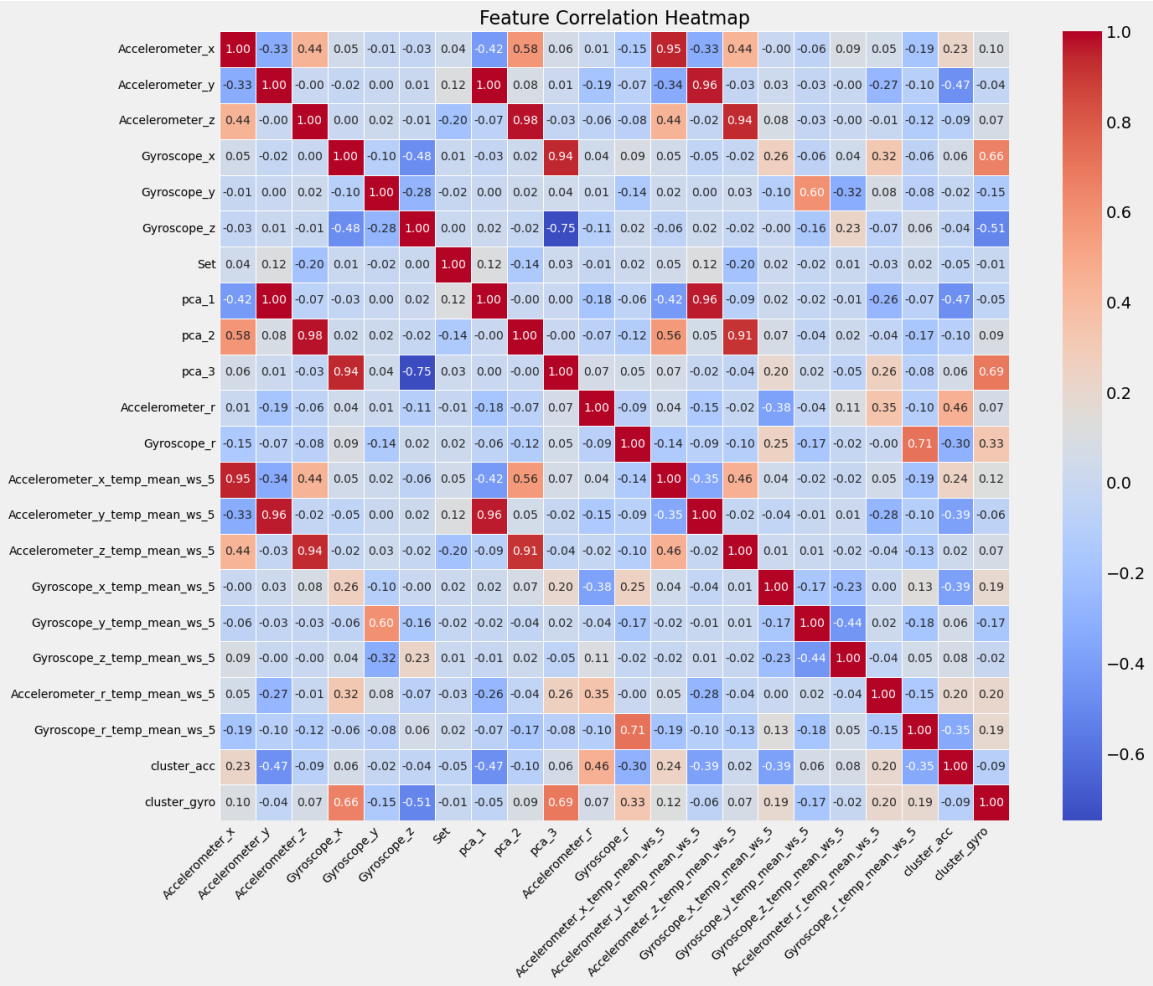

Figure 9: Pearson correlation heatmap.

Modeling Section

The number of samples for each activity was counted to understand how the data is distributed before training the model. The bar chart below shows how often each activity appears in the full dataset (*Total*), and how they are split between the training, validation and test sets.

The data was split using `train_test_split` with `stratify=y` to keep the same activity distribution in both sets. This helps the model learn from all activity types and gives a fair test.

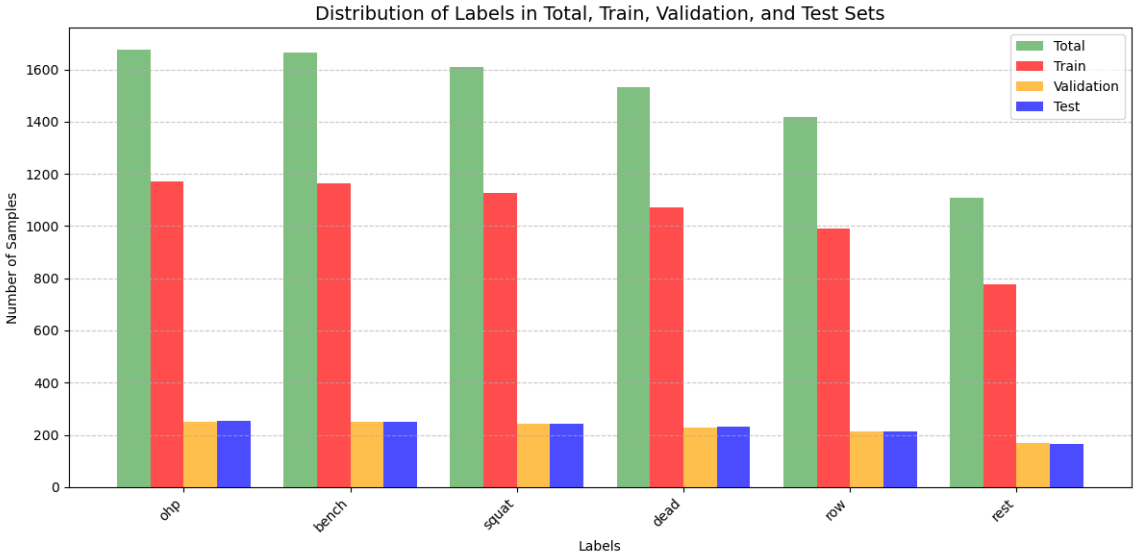

Figure 10: Distribution of activities in training and test sets.

Set Division

The features were grouped into five different sets:

- **Basic Features:** Raw accelerometer and gyroscope signals (x, y, z).
- **Square Features:** Signal magnitudes calculated from the raw sensor data.
- **PCA Features:** Features reduced using Principal Component Analysis (PCA).
- **Cluster Features:** Cluster labels from accelerometer and gyroscope data.

Mean features were initially excluded to observe how the model performs without them. Then, forward feature selection was used to identify the 10 most relevant features, which may include mean-based features.

These five sets are used to compare model performance across different feature combinations and understand the contribution of each type to classification accuracy.

Forward Selection using Decision Tree

A new set of features was selected using forward selection with a decision tree. This method adds one feature at a time, choosing the one that improves model accuracy the most at each step.

- Starts with no features.
- Each remaining feature is tested, and the one that helps the model the most is selected.

- The process repeats until the desired number of features is selected.
- The result is a list of features that improve performance step by step.

This helps identify the most important features for classification.
The final set of selected features is:

```
['Accelerometer_y_temp_mean_ws_5', 'Accelerometer_x_temp_mean_ws_5',
'Gyroscope_r_temp_mean_ws_5',
'pca_2', 'Gyroscope_z_temp_mean_ws_5', 'Gyroscope_z', 'pca_1',
'Gyroscope_x_temp_mean_ws_5', 'Accelerometer_x', 'Accelerometer_z']
```

This list includes both original sensor features and mean-based features, showing that mean values can help improve model accuracy.

Models Performance & Algorithms

DBSCAN: Outlier Detection and Parameter Tuning

DBSCAN was used to detect outliers in the dataset. To improve its performance, the key parameters were carefully studied.

Main Parameters of DBSCAN:

- **eps** (epsilon): The maximum distance between two points to be considered neighbors.
- **min_samples**: The minimum number of neighboring points required to form a dense region (a cluster).

Silhouette Score: This score measures the quality of clustering, ranging from -1 to 1.
A score closer to 1 means the points are well grouped; a score near -1 suggests poor clustering.

Parameter Tuning Process:

- The dataset is first standardized so that all features have similar scales.
- DBSCAN is applied with different combinations of `eps` (0.1 to 2.0) and `min_samples` (5 to 20).
- For each combination, the silhouette score is calculated to assess clustering quality.
- A good combination gives a high silhouette score and less than 20% of the data marked as outliers. This ensures effective clustering while minimizing data loss.

Best Parameters Found:

$$\text{eps} = 1.6, \quad \text{min\_samples} = 7$$

This combination achieved the highest silhouette score of 0.4262, indicating well-formed clusters with minimal outliers.

K-Nearest Neighbor (KNN)

KNN is a distance-based classifier that predicts a class using a majority vote among the k nearest neighbors.

KNN Tuning

KNN was tuned with `GridSearchCV` using 5-fold `StratifiedKFold` and `scoring='f1_weighted'`, so the best model was selected by weighted F1 score. A `Pipeline` with `StandardScaler` was used before KNN because distances are sensitive to feature scale.

Figure 11 compares KNN all the features sets. From Feature Set 1 to 4, training stays similar but validation drops a bit because KNN uses distances: adding extra features can add noise, so the nearest points are less truly similar and generalization gets worse. With selected features, distances are more meaningful, so validation/test go up and the gap becomes smaller.

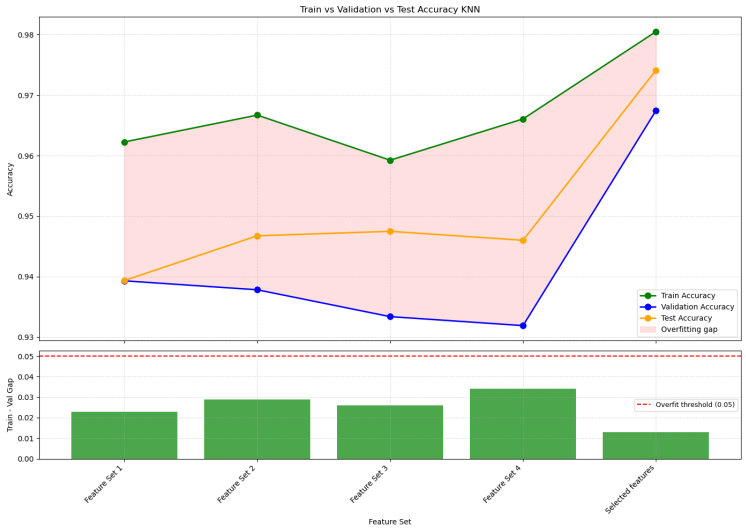

Figure 11: Knn Tuning Performance.

KNN Performance

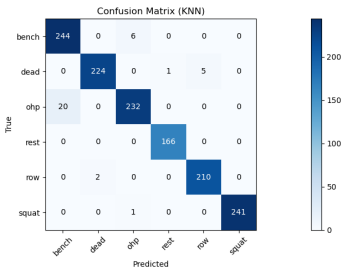

Figure 12: Best KNN performance on Validation.

With the selected features, KNN reached Training Accuracy = 0.9805, Validation Accuracy =

0.9674, and Test Accuracy = 0.9741. This means that the selected-features subset captures the most informative signals for the activities, so samples from the same class are closer in feature space and KNN can find more reliable nearest neighbors, improving generalization. In the confusion matrix, the main remaining error is that *ohp* is sometimes predicted as *bench*, showing that these two classes still have overlapping patterns for KNN.

Naive Bayes (NB)

Naive Bayes is a probabilistic classifier that assumes features are conditionally independent given the class (here we used `GaussianNB`). No hyperparameter tuning was applied, since NB usually works well with its default settings.

Figure 13 compares NB based in the same sets as knn. Performance drops from Feature Set 1/2 to Feature Set 4 because the added PCA/cluster features are likely more correlated, which violates the NB assumption and makes the probability estimates less reliable. With selected features, NB improves a lot because the subset keeps the most informative and cleaner signals, so the class distributions are easier to separate.

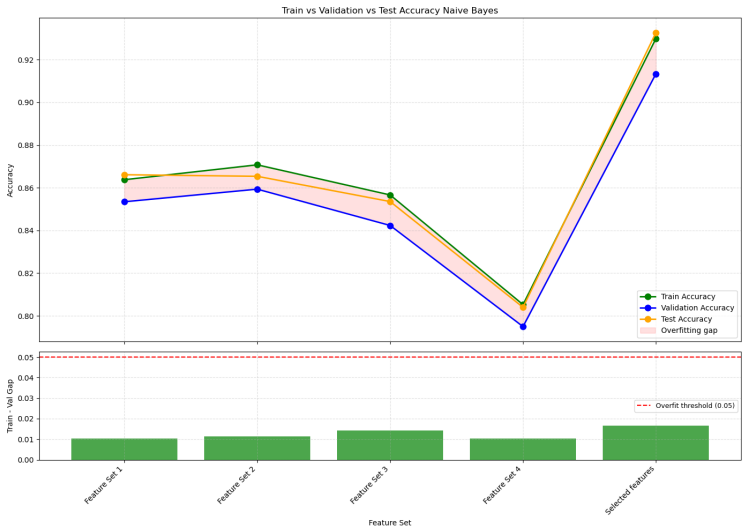

Figure 13: NB Tuning Performance.

NB Performance

Using the selected features, NB achieved Training Accuracy = 0.9299, Validation Accuracy = 0.9134, and Test Accuracy = 0.9327. From the confusion matrix, most errors happen between similar exercises, especially *ohp* predicted as *bench*, and *row* predicted as *dead*, which suggests overlapping motion patterns for these pairs.

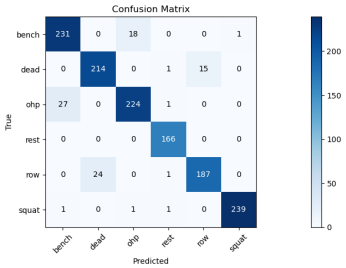

Figure 14: Confusion matrix for NB.

Logistic Regression (LR)

Logistic Regression is a linear classifier that predicts class probabilities from a weighted sum of the input features, and it uses regularization by default to reduce overfitting.

LR Tuning

`GridSearchCV` was used to tune `C` and the penalty (`l1`/`l2`) for each feature set, and the selection was based on `f1_weighted`.

In Figure 15, the selected-features set is slightly worse on training and validation than Feature Set 4. This can happen because the selected features remove some information that helps fit the training/validation split, but they reduce noise and make the model more stable, so it generalizes better to unseen test data. Also, validation and test are different splits, so small differences in sample difficulty/class mix can make the test score higher even when validation is lower.

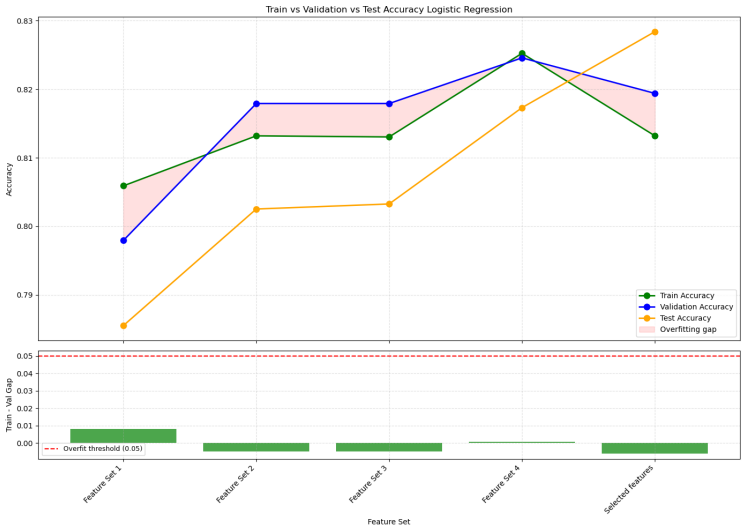

Figure 15: LR Tuning Performance.

LR Performance

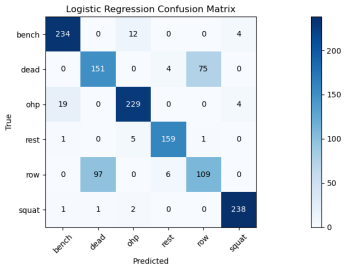

Figure 16: Cofusion matrix for LR.

With selected features, LR achieved Training Accuracy = 0.8132, Validation Accuracy = 0.8194, and Test Accuracy = 0.8284 (LR F1 = 0.8060). From the confusion matrix, the biggest confusion is between *dead* and *row* (many *dead* predicted as *row*), while *bench*, *ohp*, and *squat* are classified much better. This happens because LR can only learn linear decision boundaries, so classes with very similar sensor patterns can overlap and be harder to separate with a linear model.

Support Vector Machine (SVM) without Kernel

Linear SVM (we used `LinearSVC`) is a linear classifier that separates classes by finding a hyperplane with maximum margin, controlled by the regularization parameter `C`.

SVM Tuning

`GridSearchCV` was used to tune `C` with 5-fold cross-validation, and the selection was based on `f1_weighted` (not accuracy).

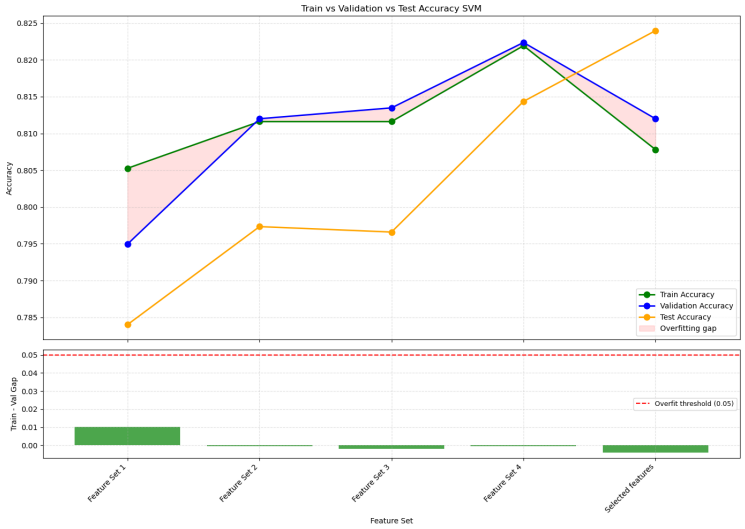

Figure 17: SVM Tuning Performance.

Figure 17 shows that SVM improves from Feature Set 1 to Feature Set 4 (Val = 0.8224), and Logistic Regression shows the same trend (Val = 0.8246 on Feature Set 4), meaning both linear models benefit from the engineered features.

For the selected features, both models lose a bit on validation compared to Feature Set 4 (SVM: 0.8120 vs 0.8224, LR: 0.8194 vs 0.8246) but gain on test (SVM: 0.8240 vs 0.8143, LR: 0.8284 vs 0.8173). This suggests the selected subset removes some split-specific patterns (hurting validation slightly) while reducing noise and improving generalization on unseen test data. Overall, LR is slightly better than SVM on the selected-features test result (0.8284 vs 0.8240), but their behavior across feature sets is very similar because both use linear decision boundaries.

SVM Performance

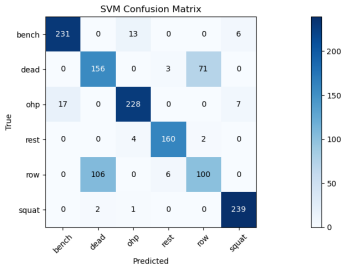

Figure 18: Confusion matrix for SVM without kernel.

The best test accuracy was achieved with the selected features (Test = 0.8240). From the confusion matrix, the biggest confusion is again between *dead* and *row*, while *bench*, *ohp*, and *squat* are classified much better. This suggests that *dead* and *row* have very similar sensor patterns, and a linear boundary is not always enough to separate them perfectly.

Decision Tree (DT)

Decision Trees create a model by splitting the data based on feature values, forming a tree structure that makes decisions step-by-step.

DT Tuning

`GridSearchCV` was used to tune the Decision Tree, and the best model was selected using `f1_weighted` to ensure balanced performance across all exercise classes.

Increasing `min_samples_leaf` requires each leaf to contain more samples. This prevents the tree from creating very small branches that only fit a few training points. As a result, the model focuses on general movement patterns instead of memorizing noise, which reduces overfitting and improves stability.

Figure 19 shows that performance improves with richer feature sets because better features allow the tree to create more meaningful split rules. The best results are obtained with the selected features (Val = 0.9275, Test = 0.9327), since this subset keeps the most relevant information while removing redundant or noisy features.

Although training accuracy is slightly higher than validation (which is expected), the strong and close validation and test results indicate that the model generalizes well to unseen data.

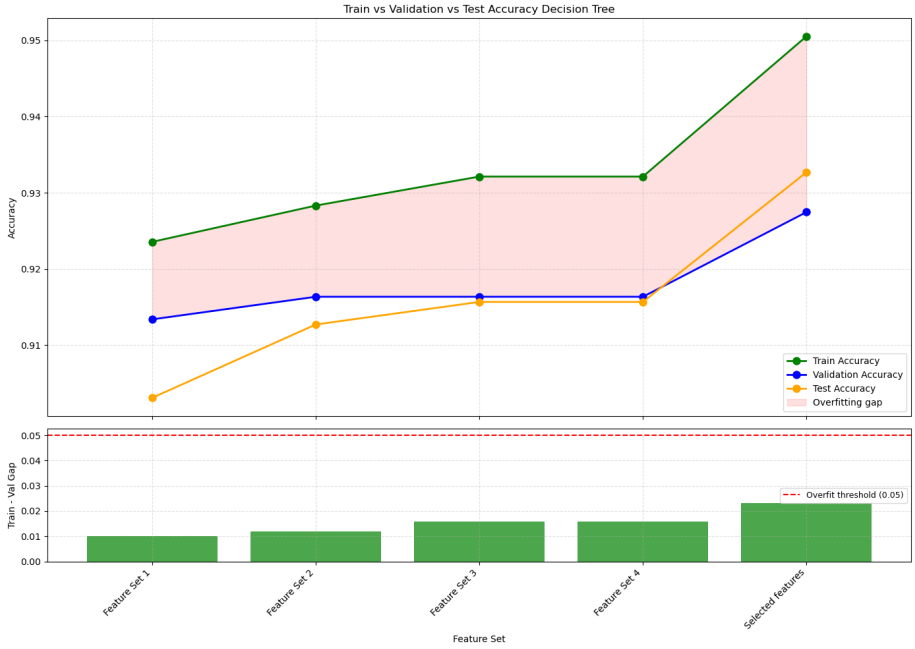

Figure 19: DT Tuning Performance.

DT Performance

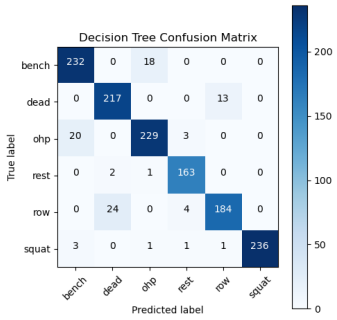

Figure 20: Confusion matrix for Decision Tree.

With selected features, DT achieved Training Accuracy = 0.9505, Validation Accuracy = 0.9275, and Test Accuracy = 0.9327 (DT F1 = 0.9335). From the confusion matrix, the main errors are between similar exercises, especially *ohp* predicted as *bench* and some confusion between *row* and *dead*. Overall, DT performs very well here because it can model non-linear feature interactions using split rules, which fits sensor patterns better than purely linear models.

Random Forest (RF)

Random Forest is an ensemble model that builds many decision trees and combines their predic-

tions. This usually improves accuracy and reduces overfitting compared to a single tree.

RF Tuning

`GridSearchCV` was used to tune the main Random Forest hyperparameters, and the best model was selected using `f1_weighted` to ensure balanced class performance. Parameters such as `max_depth` and `min_samples_leaf` control how complex each tree can grow. Limiting tree depth and requiring more samples per leaf prevents overly specific splits and reduces overfitting.

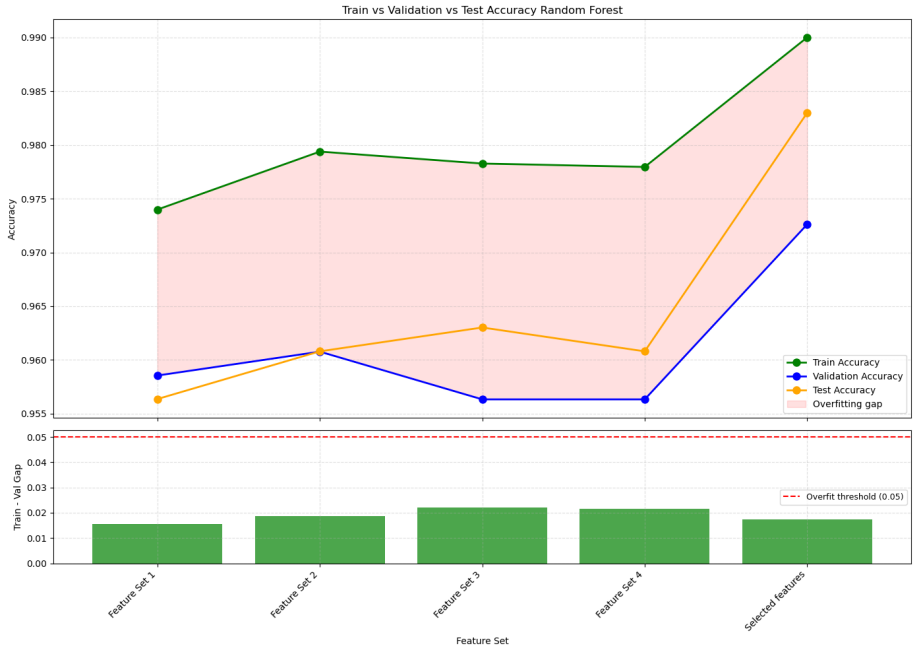

Figure 21: RF Tuning Performance.

Figure 21 shows strong and stable results across all feature sets. The best performance is achieved with the selected features (Val = 0.9726, Test = 0.9830). Although training scores are slightly higher (which is normal), validation and test results are very close, indicating good generalization.

RF Performance

With the selected features, RF achieved Training Accuracy = 0.9900, Validation Accuracy = 0.9726, and Test Accuracy = 0.9830 (RF F1 = 0.9756). This is the best overall model because combining many trees reduces sensitivity to noisy samples and captures non-linear patterns in sensor signals better than linear models.

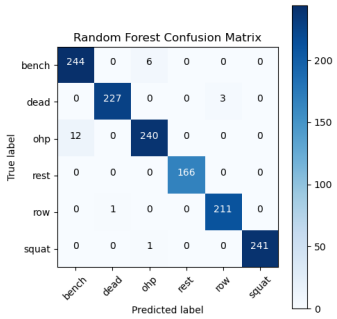

Figure 22: Confusion matrix for Random Forest.

Models Comparison

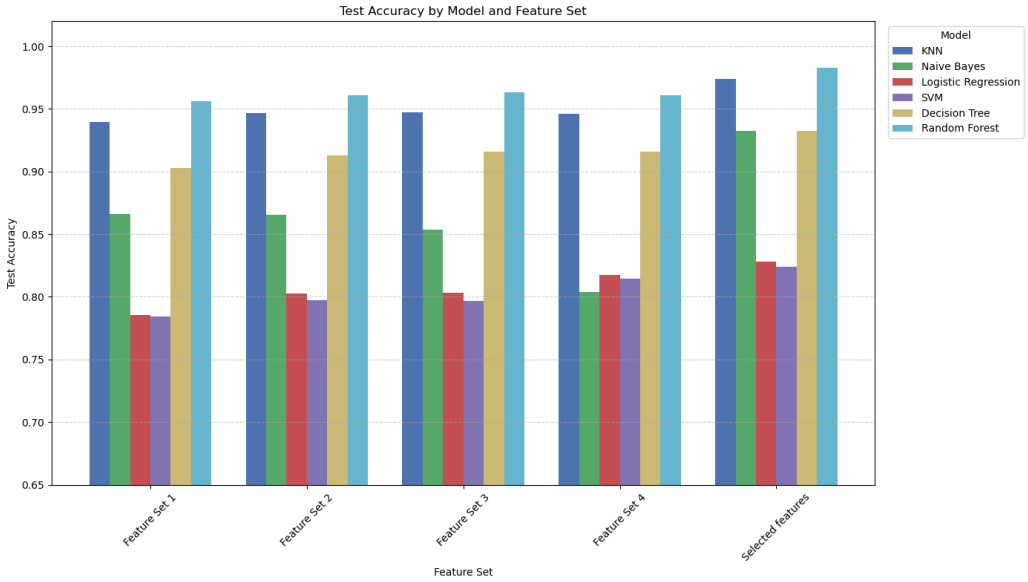

Figure 23: Models Results.

Figure 23 shows that non-linear models perform better than linear ones for this task.

Random Forest is the best model with Training = 0.9900, Validation = 0.9726, and Test = 0.9830 (F1 = 0.9756). It combines many trees, so it handles complex and noisy sensor patterns very well. It performs strongly on all exercises, especially *squat*, *dead*, and *rest*.

KNN also performs very well (Test = 0.9741). With selected features, samples from the same exercise are closer in feature space. Small errors remain between similar movements like *ohp* and *bench*.

Decision Tree reaches Test = 0.9327 (F1 = 0.9335). It models non-linear patterns, but since it is a single tree, it is less stable than Random Forest.

Naive Bayes also achieves Test = 0.9327. It works well after feature selection, but struggles when exercises have similar signal distributions (e.g., *row* and *dead*).

Linear models perform worse: **Logistic Regression** (Test = 0.8284, F1 = 0.8060) and **Linear SVM** (Test = 0.8240). They use linear decision boundaries, so they cannot clearly separate very similar exercises like *row* and *dead*.

	Precision						Recall						F1-Score					
Model	Decision Tree	Knn	Logistic Regression	Naive Bayes	Random Forest	SVM	Decision Tree	Knn	Logistic Regression	Naive Bayes	Random Forest	SVM	Decision Tree	Knn	Logistic Regression	Naive Bayes	Random Forest	SVM
Class																		
bench	0.910	0.924	0.918	0.892	0.953	0.931	0.928	0.976	0.936	0.924	0.976	0.924	0.919	0.949	0.927	0.908	0.964	0.928
dead	0.893	0.991	0.606	0.899	0.996	0.591	0.943	0.974	0.657	0.930	0.987	0.678	0.918	0.982	0.630	0.915	0.991	0.632
ohp	0.920	0.971	0.923	0.922	0.972	0.927	0.909	0.921	0.909	0.889	0.952	0.905	0.914	0.945	0.916	0.905	0.962	0.916
rest	0.953	0.994	0.941	0.976	1.000	0.947	0.982	1.000	0.958	1.000	1.000	0.964	0.967	0.997	0.949	0.988	1.000	0.955
row	0.929	0.977	0.589	0.926	0.986	0.578	0.868	0.991	0.514	0.882	0.995	0.472	0.898	0.984	0.549	0.903	0.991	0.519
squat	1.000	1.000	0.967	0.996	1.000	0.948	0.975	0.996	0.983	0.988	0.996	0.988	0.987	0.998	0.975	0.992	0.998	0.968

Figure 24: Precision, Recall and F1-Scores per Class.

The table confirms these results.

Random Forest has the highest and most balanced precision, recall, and F1-scores (mostly 0.96–1.00). This means very few false positives and false negatives.

KNN and **Decision Tree** also show strong and stable scores (mostly above 0.90).

Rest and *squat* are the easiest exercises (F1 close to 1.00 for most models). *Bench* and *ohp* are slightly harder but still well classified. *Row* and *dead* are the hardest, especially for linear models, because their sensor patterns are very similar.

Overall, non-linear and ensemble models work better because gym movements are complex and not linearly separable.

Conclusion

This project focused on analyzing raw accelerometer and gyroscope data by extracting important features like mean and magnitude and applying filters to clean the signals. Several machine learning models were tested to classify different gym exercises. Among them, Random Forest performed best due to its ability to handle complex and noisy data. The project showed that careful data preparation and feature extraction play a crucial role in improving model accuracy. Overall, the work demonstrates the importance of each step in the machine learning process when working with sensor data, from understanding the signals to selecting and tuning models.

Bibliography

- Shoaib, M., Bosch, S., Incel, O. D., Scholten, H., & Havinga, P. J. M. (2021). Complex Human Activity Recognition Using Smartphone and Wrist-Worn Motion Sensors. *IEEE Sensors Journal*, 21(12), 13352–13363. `https://ieeexplore.ieee.org/document/9430501`
- Mukherjee, S., & Sharma, N. (2021). Machine Learning Approaches for Human Activity Recognition: A Comparative Study. *arXiv preprint*. `https://arxiv.org/abs/2109.01118`
- Wikipedia. (n.d.). Gyroscope. `https://en.wikipedia.org/wiki/Gyroscope`
- Wikipedia. (n.d.). Fitness tracker. `https://en.wikipedia.org/wiki/Fitness_tracker`